

Assignment 13

The main objective in this assignment is to get a basic understanding on formal transaction-based tracking AIP denoted as formal scoreboards. Formal scoreboards are an abstraction technique to verify the correct propagation of data from the inputs to the outputs of a design or component of a design. For that reason, scoreboards are, typically, employed for black box checks. In this assignment, you will create a formal scoreboard and use it to formally verify a FIFO.

Related Theory

Formal verification can quickly consume a substantial quantity of computational resources, such as main memory or runtime. Consequently, employing ad-hoc formal verification is only feasible for small designs with a relatively small state space. Larger designs with a medium or large state space require a more sophisticated verification approach to prevent that checks need an infeasible amount of computational resources. A central part of such a formal verification flow is the application of abstraction techniques, like formal scoreboards, which can significantly reduce the computational complexity of formal checks.

The fundamental concept of the scoreboard technique is to model the propagation through the design under verification (DUV) only for a single data packet, rather than for all data packets that might be concurrently processed in the DUV. By removing the data of other packets from consideration significantly reduces the amount of registers required for the formal model and, therefore, reduces the complexity of this model.

A formal scoreboard provides the following checks:

- Data integrity
- Packet order
- Packet loss
- Packet duplication

In order to verify the correct order of packets and to ensure that no packet is lost or duplicated, the scoreboard tracks the number of packets that entered the DUV before the packet under consideration. For example, when five packets enter the DUV before the considered sixth packet, then six packets must leave the DUV and the sixth packet that leaves the DUV must have the expected data value. The longest analysis window for a formal check is, therefore, equal to the maximum number of cycles that a data packet can reside inside the DUV.

The selection of the data packet is unconstrained, allowing the formal tool to select an arbitrary packet from an arbitrary sequence of packets. This ensures that no propagation-related bug can escape the formal checks. This means, that in the example above, the formal will perform the same checks for the first five packets and any number of packets that can come after the sixth packet.

The interface of a formal scoreboard resembles the interface of a FIFO. The verification engineer has to provide the information when a data packet enters the DUV (**push**), when a data packet leaves the DUV (**pop**) and what data value the leaving packet should have (**data_in**) and what data value the leaving packet actually has (**data_out**). Figure 1 shows how a scoreboard is bound to the DUV.

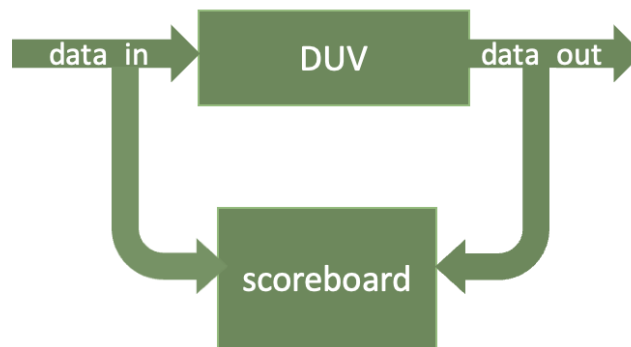


Figure 1: Scoreboard binding

When this information requires some computation and cannot be directly obtained from existing design signals, then best practice is to declare a signal for each primary input and place their combinationally computation before the instantiation of the formal scoreboard.

We suggest to implement the storage of the considered data packet by using the non-deterministic approach outlined in Part 3 Section 4 of the theory document

Design description

Figure 2 depicts the block diagram of the considered FIFO buffer design.

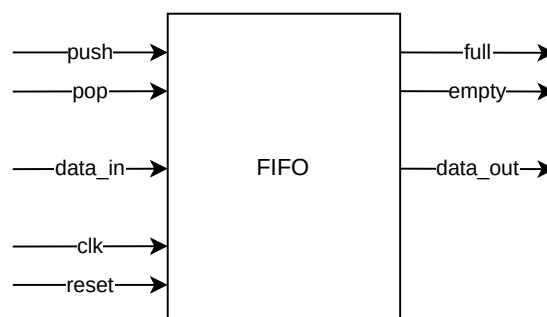


Figure 2: FIFO block diagram

The design has the following ports:

data_in: Data to be written to the FIFO.
data_out: Data to be read from the FIFO.
full: When asserted, then all buffer slots are used and no pushes are allowed.
push: Tells the FIFO to read the value at **data_in** and store it in the internal buffer unless the FIFO is full.
empty: When asserted, then all buffer slots are free and no pops are allowed.
pop: Tells the FIFO to load the next data value from the internal buffer and write it to **data_out** unless the FIFO is empty.

Furthermore, the FIFO has the following parameters:

WIDTH: Defines the bitwidth of the data values. The allowed value range is $1 \leq \text{WIDTH}$.
DEPTH: Defines the number of data slots. The allowed value range is $1 \leq \text{DEPTH}$.

After reset the buffer of the FIFO should be empty and all output signal should be assigned accordingly.

Task 13.1

In this first task you will create a standard ad-hoc assertion. The performance of this assertion serves as a baseline against which you can compare the performance of the scoreboard you will create in the following tasks of this assignment.

Formulate the assertion **data_integrity** to check that when data is pushed into the FIFO, the same data will eventually be popped out.

Place this assertion inside the scoreboard file and let the formal tool verify the data integrity.

Hint: In the event that the proof does not converge, try to reduce the parameters width and depth to lower values.

Task 13.2

In this task you will create auxiliary logic necessary for a formal scoreboard. The assertions created in the following task will use this logic to formally verify the correct behaviour of the FIFO.

Please implement the following central parts of a formal scoreboard as logic inside the scoreboard file. We intentionally provide only an abstract description instead of concrete SystemVerilog code. This provides a certain degree of freedom in how you implement the formal scoreboard and should help you to really understand the basic principles of this technique.

Sampling In

Create the single bit signal `fv_sampled_in` which, when asserted, should signal that the scoreboard started to track a data packet. The condition for this situation is that a data packet entered the DUV and that this packet was selected by the formal tool to be tracked. The packet selection is modeled by creating the undriven signal `fv_arbit_window` and using it in the condition. The data packet selected to be tracked should be stored in the signal `fv_sampled_data`.

Packet Counter

Now create the signal `fv_tracking_counter` which should be incremented for each data packet entering the DUV and should be decremented for each packet leaving the DUV. However, the incrementation should only be performed for every entering packet before the packet to be track was sampled-in. Likewise, the decrementation should only be performed for every leaving packet before the tracked data packet was sampled-out.

Hint: The tracking counter should consider the case where one data packet enters and one data packet leaves the DUV in the same clock cycle.

Sampling Out

Create the single bit signal `fv_sampled_out` which, when asserted, should indicate that the value of the data packet currently leaving the DUV must be equal to the tracked (sampled-in) data packet. The condition for this is situation is that the tracking counter has the value 1 and that data leaves the DUV.

Hint: Create a signal and compute the condition combinational, because it can be re-used in the next task.

Task 13.3

In this task, you will create the assertions that use the auxiliary logic created in the previous task to perform the formal scoreboard checks.

Create the following assertions inside the scoreboard file.

- Formulate the property `data_ordering_and_integrity` to check that when the tracked data packet is leaving the DUV, then, the data leaving the DUV must be equal to the tracked data. This assertion checks for data duplication, loss, corruption and also ordering.
- Formulate the property `data_in_equal_data_out` that sampled-in data will, eventually, be sampled-out. This assertion checks for data loss, but doesn't check the data order.

- Formulate the assumption `data_not_in_then_never_at_output` that when a data value is not sampled-in, then it cannot be sampled-out. This assumption is an invariant guiding the formal analysis.
- Formulate the assumption `tracking_counter_not_zero_if_data_in` that when data was sampled-in and is not, yet, sampled-out, then the tracking counter is not zero. This assumption is an invariant guiding the formal analysis.
- Add the following cover properties `cover_sampled_in` and `cover_sampled_out` to ensure that an asserted `fv_sampled_in` and `fv_sampled_out` are reachable. The covers help to identify mistakes in the auxiliary logic and the binding of the scoreboard to the DUV. They are essential, because these signals regularly appear in the antecedent of an assertion.

Load design and AIP with the tcl script for your formal verification tool and run all assertions. Fix any issue you encounter.

How do the scoreboard assertions perform in comparison with the ad-hoc assertion from the first task? How do the assertions perform when you increase the parameter values of the FIFO?

Task 13.4

When all assertions are holding and everything looks good, then take a step back and think about the features that are supported by the FIFO. I.e., do a completeness analysis.

- Is the full FIFO behaviour covered by the scoreboard?
- Provide an argument for every feature which you think is covered.
- Provide ideas to cover every feature which you think is not covered.

Hint: Do you know the bypass feature of FIFOs?